Date: 21/10/26 Assessement tool-3 Name: R.Namitha

Day: Wednesday CO_2 Regno: 192412080

Debugging & optimization

1. perceptron Fails to converge

Definition: A perceptron is a simple neural network used for classification. For linearly separable data it should converge if the standard perceptron learning rule is implemented

Debugging steps:

* check the training data and class labels
* verify that the dataset is linearly separable
* check the weight and bias update formula
* print weights, bias, classification errors and iteration
* Test the algorithm on a small, simple dataset

Optimization techniques:

- use a suitable learning rate
- Normalize input feature
- Correctly update all samples are classified correctly

Weight update formula:

$$w_{new} = w_{old} + \eta (y - \hat{y}) x$$

* Correct implementation, proper bias handling and suitable preprocessing help the perceptron.

2. Slow convergence in a multilayer perceptron

Definition: A multilayer perceptron (MLP) is a neural Network containing, input, hidden, and output layers.

* Vanishing gradients: Gradients become very small, so weights update slowly
* poor activation function: Sigmoid or tanh may saturate and slow learning in deep networks
* poor weight initialization: Improper initial weights may make training difficult.
* Learning rate problems: A very low rate slows learning while a high rate may cause unstable training
* Local minima or saddle points: optimization may stop making usefull progress

Debugging steps:

1. Check training and validation loss
2. monitor gradient in each layer

Optimization techniques:

- use ReLU activation in hidden layers
- use Xavier or the weight initialization

3. Genetic Algorithm produces poor solutions

Definition: A genetic Algorithm is an optimization technique inspired by natural selection. It improves solution using selection, crossover

* The chromosome may not represent the solution quality properly
* The fitness function may not measure solution quality properly

Debugging steps:

* check chromosome representation
* verify the fitness calculation
* monitor the best and average fitness
* check population diversity
* Run the algorithm multiple times using different random seeds

optimization techniques:

* Adjust the mutation and crossover rates
* use tournament selection or suitable
* maintain a sufficiently diverse
* use elistism to preserve the best solutions
* Increase population size if necessary
* Set a suitable stopping condition.

4. Deep NN With genetic programming : over fitting and high cost

Definition: A deep NN learns complex patterns, while genetic programming searches for suitable programs or model structure. combining them can increase computational cost

Overfitting: The model memorize training data but performs poorly on new data

over-parameterization: Too many weights and model components increase complexity

Debugging steps:

1. Compare training loss with validation loss
2. check the number of layer
3. measure training time and memory usage
4. Identify unnecessary model components.

optimization techniques:

* Add penalty to discourage unnecessarily large weight
* Randomly disable some neurons during training
* Remove unnecessary weights, neurons, or branches
* stop training when validation
* Regularization, pruning, and efficient candidate and computational cost while improving.

5. Decision tree has high training Accuracy but poor test Accuracy

Definition: A decision tree is a Supervised learn algorithm that makes predictions using a sequence of feature based decisions

* Overfitting: The tree memorizes training example
* Excessive depth: Too many branches capture noise instead of general patterns
* Noisy data: Incorrect labels and irrelevant features reduce performance

* Debugging steps:
  1. Compare training accuracy and test accuracy
  2. Examine the tree depth and number
  3. check missing values, noise and incorrect
  4. Use cross - validation to estimate

- optimization techniques:
* Remove branches that do not improve generalization
* prevent excessive splitting
* Require sufficient examples in each leaf
* Select suitable tree parameters using multiple data splits.